

Surgical Knowledge Swapping in Large Language Models: Causal Localization and Precision Model Editing

CSCI 544 — Project Midterm Report, Group 34

Abhishek Soundalgekar
soundalg@usc.edu

Aryan Sawant
aryanavi@usc.edu

Prathiksha Nijamkari
ravibabu@usc.edu

Rhesha Vinod
rvinod@usc.edu

Siddhanth Salian
ssalian@usc.edu

Abstract

Large Language Models encode factual knowledge in parameters learned during pretraining. When facts change, full retraining is expensive and risks catastrophic forgetting. This report presents our completed work on *surgical knowledge swapping*—targeted modification of factual associations in pretrained transformers—comparing ROME, MEMIT, and LoRA on Qwen2.5-1.5B-Instruct using the CounterFact benchmark (Meng et al., 2022). All five member tasks are complete. Causal tracing identified Layer 15 as the critical memory layer. ROME achieved perfect Edit Success Rate (ESR = 1.00) with strong paraphrase generalization (PG = 0.80). MEMIT matched ROME’s ESR while achieving superior locality (LL = 1.00 vs. 2.61) and zero sequential interference (IR = 0.00). LoRA, serving as the fine-tuning baseline, exhibited lower ESR (0.40) and substantially higher locality leakage (LL = 52.34), confirming that gradient-based editing methods offer a fundamentally better precision–locality trade-off than adapter fine-tuning for this task.

1 Tasks Performed

1.1 Task 1: Environment Setup, Inference, and Reproducibility

Model Loading: We load Qwen/Qwen2.5-1.5B-Instruct (Qwen Team, 2025) (1,544M parameters) in 4-bit NF4 quantization via bitsandbytes on CUDA, or in float16 on Apple MPS when CUDA is unavailable. Our proposal targeted a 7B model but VRAM constraints required the 1.5B variant; the pipeline scales to larger models with a single config change.

Tokenizer and Inference: The AutoTokenizer uses EOS as the pad token. A reusable generate_text() function performs greedy decoding and returns both the generated text and wall-clock latency. Setting max_new_tokens = 10

resolved early run-on outputs that listed multiple facts instead of stopping at one answer.

Reproducibility: Seeds are fixed across torch, numpy, and random (seed = 42). A central CONFIG dictionary stores all hyperparameters and logs every prompt–output pair alongside its configuration snapshot via a run_log.jsonl file. Batch testing confirmed deterministic behaviour across repeated runs.

1.2 Task 2: Causal Tracing and Activation Patching

We identify which MLP layer stores a given fact—a prerequisite for ROME (Meng et al., 2022) and MEMIT (Meng et al., 2023)—using the three-run causal tracing procedure across all 28 transformer layers.

Clean Run: A factual prompt (e.g., “The capital of France is”) is passed through the model and all 28 layer hidden states are cached (shape: $[1, T, 1536]$ per layer).

Corrupted Run: Rather than text-level prompt modification (which causes tensor shape mismatches), we inject Gaussian noise ($\sigma=0.1$) directly into transformer layer outputs, keeping input dimensions unchanged.

Activation Patching: For each layer $l \in \{0, \dots, 27\}$, we restore its clean activation in the fully corrupted run and measure recovery of the correct token probability $P(\text{target} \mid \text{prompt})$. The layer with the highest recovery is designated the **critical memory layer**.

Outcome: Layer 15 achieved the highest patching recovery score (116.5% of the clean baseline), confirming it as the primary factual memory layer in Qwen2.5-1.5B-Instruct (Geva et al., 2020). Three implementation issues were resolved during this task: (1) GPU OOM on the 7B model → switched to 1.5B; (2) shape mismatches with text-level corruption → activation-level noise injection; (3) KeyError: layer_27 from stale hooks

→ explicit hook clearing before each run. All subsequent editing tasks target Layer 15.

1.3 Task 3: ROME Implementation and Single-Edit Experiments

We implement ROME (Meng et al., 2022) as a rank-one update to the down_proj weight matrix at Layer 15. For compatibility with 4-bit quantized (Params4bit) weights, the update is applied via a registered PyTorch forward hook rather than direct weight mutation, which is mathematically equivalent:

$$W_{\text{new}} = W + \frac{(v^* - v_0) \hat{k}^\top}{\hat{k}^\top \hat{k}}, \quad \hat{k} = \frac{k}{\|k\|^2} \quad (1)$$

where k is the pre-down_proj activation at the subject’s last token position, v_0 the corresponding current output, and v^* the target value vector found by 25 steps of Adam gradient descent (lr=0.4) minimising cross-entropy loss on the new target token.

Dataset: We evaluate on 5 entries sampled from the CounterFact benchmark (Meng et al., 2022) (21,919 total entries), downloaded from the official ROME project server. No hand-crafted facts are used anywhere in the pipeline.

Results: Each edit is evaluated in isolation (model reset between edits). Table 1 shows per-fact results.

Subject	New	ESR	PG	LL
Danielle Darrieux	English	1.0	1.0	1.000
Edwin of Northumbria	Islam	1.0	1.0	0.990
Toko Yasuda	piano	1.0	0.5	9.082
Auton. Univ. Madrid	Sweden	1.0	1.0	1.000
Lyon	Manila	1.0	0.5	1.000
Mean		1.000	0.800	2.614
Std		0.000	0.274	3.616

Table 1: ROME single-edit results on 5 CounterFact entries. ESR: Edit Success Rate (hard, generation-based). PG: Paraphrase Generalization. LL: Locality Leakage ratio (lower is better).

ROME achieves perfect ESR (1.00) and strong paraphrase generalization (PG=0.80). Locality leakage is elevated on two edits (Toko Yasuda: LL=9.08) due to the single-layer, single-pass nature of the rank-one update.

1.4 Task 4: MEMIT Implementation and Batch/Sequential Edits

We implement MEMIT (Meng et al., 2023), which extends ROME by distributing rank-one updates across a contiguous range of MLP layers $\mathcal{L} = \{13, 14, 15, 16, 17\}$ with per-layer scale

$s = 1/|\mathcal{L}| = 0.2$. All five edits are applied simultaneously in a single batch pass.

Batch Edit Results: MEMIT matches ROME’s perfect ESR (1.00) while substantially reducing locality leakage (mean LL=1.00 vs. ROME’s 2.61), confirming that distributing updates across multiple layers spreads the per-layer perturbation and better preserves unrelated knowledge.

Sequential Interference Analysis: We apply edits one at a time and after each addition measure whether all previously applied edits still hold (ESR evaluated on accumulating prefix). Results show **zero interference** (IR=0.00) at every step: each new edit leaves all prior edits intact, demonstrating strong stability even under sequential accumulation on this 5-edit benchmark.

After edit #	Prior edits broken	IR
2	0 / 1	0.00
3	0 / 2	0.00
4	0 / 3	0.00
5	0 / 4	0.00
Average		0.00

Table 2: MEMIT sequential interference. IR: fraction of prior edits broken by the most recent edit addition.

1.5 Task 5: LoRA Baseline, Metrics Framework, and Visualizations

We implement a LoRA (Hu et al., 2021) fine-tuning baseline using rank $r=8$, $\alpha=16$ adapters attached to the down_proj layers at $\mathcal{L} = \{13, 14, 15, 16, 17\}$, matching MEMIT’s target layers for a fair comparison. Only adapter matrices $A \in \mathbb{R}^{r \times d_{\text{in}}}$ and $B \in \mathbb{R}^{d_{\text{out}} \times r}$ are trained; base weights remain frozen. This introduces **419,840 trainable parameters**—just 0.027% of the model’s 1.544B total parameters.

Training: We fine-tune for 4 epochs (lr = 5×10^{-4} , AdamW) with prompt-masked cross-entropy loss (only target tokens supervised). Training loss decreases from 10.10 (epoch 1) to 3.98 (epoch 4), confirming successful adapter learning.

Results: LoRA achieves ESR=0.40 and PG=0.20, markedly lower than ROME and MEMIT. More critically, locality leakage is extreme (mean LL=52.34), confirming that even parameter-efficient adapter fine-tuning cannot match the surgical precision of gradient-based rank-one editing methods for this task.

Comprehensive Metrics Framework: Table 3 presents the complete cross-method comparison

across all five metrics. ROME wins on ESR and PG; MEMIT wins on locality (LL) and sequential stability (IR); LoRA performs worst on all metrics for this knowledge-editing setting.

Method	ESR $\uparrow$	PG $\uparrow$	LL $\downarrow$	CS $\uparrow$	IR $\downarrow$
ROME	1.000	0.800	2.614	0.200	—
MEMIT	1.000	0.200	1.000	0.200	0.000
LoRA	0.400	0.200	52.337	0.200	—

Table 3: Full metric comparison. ESR: Edit Success Rate (hard). PG: Paraphrase Generalization. LL: Locality Leakage (lower is better). CS: Consistency Score. IR: Interference Rate (MEMIT sequential only). Arrows indicate preferred direction. **Bold** = best per column.

2 Challenges Encountered and Resolutions

Compute and Memory: ROME and MEMIT require per-layer activation capture, multiple forward passes, and iterative gradient optimisation per edit. We resolved memory pressure by deleting intermediate activation tensors immediately after use and running all development on the 1.5B model; GPU-intensive runs use CUDA (4-bit NF4) or Apple MPS (float16).

4-Bit Quantization Compatibility: Direct weight access on Params4bit objects (bitsandbytes) fails because `.weight.shape` is not defined. We resolved this by using `in_features/out_features` for LoRA adapter initialization, and applying ROME/MEMIT updates via forward hooks rather than weight mutation—maintaining full mathematical equivalence.

CounterFact Dataset Availability: The EleutherAI/counterfact Hugging Face dataset was removed from the Hub mid-project. We resolved this by downloading directly from the ROME paper’s official server (rome.baulab.info/data/dsets/counterfact.json) with local caching, eliminating any dependency on Hugging Face availability.

Generalization vs. Locality Trade-off: ROME’s single-layer update shows elevated locality leakage on some edits (e.g., LL=9.08 for Toko Yasuda), while MEMIT’s multi-layer distribution consistently achieves LL $\approx$ 1.00. This empirically validates the theoretical motivation for MEMIT’s design.

LoRA Locality Leakage: Global adapter updates across multiple layers bleed into unrelated token distributions (mean LL = 52.34). This is an inherent

limitation of fine-tuning-based approaches and not resolvable without architectural changes such as subject-conditioned gating.

3 Individual Contributions

Rhesha Vinod (Tasks 1 & 2): Built the inference pipeline, cross-platform model loading (CUDA/MPS/CPU), `generate_text()` with timing, CONFIG logging with JSONL output, batch testing, and the clean-run hidden-state caching pipeline for causal tracing.

Prathiksha Nijamkari (Tasks 1 & 2): Implemented reproducibility infrastructure, the Gaussian noise-based corrupted run, the 28-layer activation patching loop with hook cleanup, and resolved the `KeyError: layer_27 stale-hook` bug that confirmed Layer 15 as the critical memory layer (116.5% recovery).

Abhishek Soundalgekar (Task 3): Implemented the full ROME pipeline: subject token localisation, key vector k and value v_0 extraction via forward hooks, 25-step Adam value optimisation, hook-based rank-one update (4-bit safe), single-edit experiments on 5 CounterFact entries, and ESR/PG/LL evaluation (ESR = 1.00, PG = 0.80).

Aryan Sawant (Task 4): Implemented the MEMIT multi-layer batch architecture with per-layer scale $s=0.2$, applied updates across layers 13–17, ran batch and sequential edit experiments, and computed the interference rate (IR = 0.00 across all 4 sequential steps), confirming zero edit interference.

Siddhanth Salian (Task 5): Implemented the LoRA baseline (LoRALinear with safe `in_features/out_features` initialisation, 419,840 trainable params, 0.027% of total), 4-epoch AdamW fine-tuning with loss curve logging, the full five-metric framework (ESR, PG, LL, CS, IR), and all comparative visualizations including the 6-panel final figure.

References

- Mor Geva, Roei Schuster, Jonathan Berant, and Omer Levy. 2020. Transformer feed-forward layers are key-value memories. *arXiv preprint arXiv:2012.14913*.
- Kevin Meng, David Bau, Alex Andonian, and Yonatan Belinkov. 2022. Locating and editing factual associations in GPT. *arXiv preprint arXiv:2202.05262*.
- Kevin Meng, Arnab Sharma, David Bau, and Yonatan Belinkov. 2023. Mass editing memory in a transformer. *arXiv preprint arXiv:2210.07229*.
- Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2021. LoRA: Low-rank adaptation of large language models. *arXiv preprint arXiv:2106.09685*.
- Damai Dai, Li Dong, Yaru Hao, Zhifang Sui, Baobao Chang, and Furu Wei. 2021. Knowledge neurons in pretrained transformers. *arXiv preprint arXiv:2104.08696*.
- Qwen Team. 2025. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*.